Practical 1

Aim

- To write a program to solve the 2-water jug problem by manually providing moves from the user.
- To write a program to implement Missionaries and Cannibals problem.

Theory

The Water Jug problem and the Missionaries and Cannibals problem are classical problems in Artificial Intelligence that help in understanding state representation, constraints, and problem-solving techniques.

In the Two Water Jug problem, there are two jugs of fixed capacities, one of 5 litres and the other of 3 litres, without any measurement markings. The objective is to obtain exactly 4 litres of water using a limited set of operations such as filling a jug completely, emptying a jug completely, and pouring water from one jug to another. The state of the system is represented using a pair of values indicating the amount of water present in each jug. In this experiment, the user manually provides the moves, which allows better understanding of how each operation changes the current state. This approach emphasizes logical reasoning rather than automated search and helps in visualizing how intermediate states lead to the final goal.

The Missionaries and Cannibals problem is a constraint-based problem that involves safely transferring three missionaries and three cannibals across a river using a boat that can carry a maximum of two people. The main constraint is that missionaries should never be outnumbered by cannibals on either bank, as this would lead to an unsafe state. The problem is represented using

states that indicate the number of missionaries, cannibals on the left bank, and the position of the boat. To solve this problem, Breadth First Search (BFS) is used to explore all possible valid and safe states systematically. Invalid states that violate the given constraints are discarded. This problem demonstrates the importance of state validation, constraint handling, and systematic exploration to reach a safe goal state.

Algorithm

2-Water Jug Problem

1. Start by initializing both jugs with zero litres of water.
2. Define the capacities of the two jugs as 5 litres and 3 litres and set the target amount as 4 litres.
3. Display a menu showing all allowed operations such as filling a jug, emptying a jug, and pouring water from one jug to another.
4. Accept the user's choice of operation through manual input.
5. Perform the selected operation and update the current water levels in both jugs accordingly.
6. After every operation, check whether the target amount is present in any one of the jugs.
7. If the target amount is obtained, display a success message and stop execution.
8. Otherwise, repeat the process until the goal is achieved or the user exits the program.

Missionaries and Cannibals Problem

1. Represent each state as a tuple containing the number of missionaries, cannibals on the left bank, and the position of the boat.
2. Define the initial state as all missionaries and cannibals on the left bank with the boat also on the left side.
3. Create a queue and insert the initial state to begin Breadth First Search.
4. Remove one state at a time from the queue and check whether it is the goal state.
5. Generate all possible next states based on valid boat movements.
6. Check safety conditions using the validity function.

7. Ignore invalid or already visited states to avoid repetition.
8. Add only valid and unvisited states to the queue for further exploration.
9. Continue the process until the goal state is reached.

Dataset Description

Source:
The input for both experiments is user-defined and generated during program execution.

Attributes:
For the Two Water Jug problem, the attributes include the capacities of the two jugs, the current amount of water present in each jug, and the target quantity to be measured.
For the Missionaries and Cannibals problem, the attributes include the number of missionaries and cannibals on the left bank of the river and the position of the boat.

Size:
No external dataset is used in these experiments. The number of possible states is limited and finite, as all states are generated based on predefined rules and constraints.

Implementation

2-Water Jug Problem

```
# Two Water Jug Problem

# Jug X = 5 litres, Jug Y = 3 litres

# Target 4 litres in any jug



jugX = 0     # water in 5 litre jug

jugY = 0     # water in 3 litre jug
```

```
capX = 5

capY = 3

target = 4


print("Target is to get", target, "litres in any one jug")

while True:


    print("\nCurrent State:")

    print("Jug X:", jugX, "litres")

    print("Jug Y:", jugY, "litres")


    # check goal in any jug

    if jugX == target or jugY == target:

        print("Goal achieved")

        break

    # menu for user to select operation

    print("\nSelect Operation:")

    print("1. Fill Jug X")

    print("2. Fill Jug Y")

    print("3. Empty Jug X")

    print("4. Empty Jug Y")
```

```
print("5. Pour Jug X into Jug Y")

print("6. Pour Jug Y into Jug X")

print("7. Exit")


choice = int(input("Enter choice: "))


if choice == 1:

    jugX = capX


elif choice == 2:

    jugY = capY


elif choice == 3:

    jugX = 0


elif choice == 4:

    jugY = 0


# pouring water from jug X to jug Y

elif choice == 5:

    free_space = capY - jugY
```

```
        moved = min(jugX, free_space)

        jugX = jugX - moved

        jugY = jugY + moved

    # Jug Y to Jug X

    elif choice == 6:

        free_space = capX - jugX

        moved = min(jugY, free_space)

        jugY = jugY - moved

        jugX = jugX + moved

    elif choice == 7:

        print("Program terminated by user")

        break

    else:

        print("Invalid option selected")
```

Missionaries and Cannibals

```
from collections import deque

# function to check whether the given state is safe or not

def validity(state):

    m, c, boat = state
```

```
    # missionaries and cannibals count should be between 0 and 3
    if not (0 <= m <= 3 and 0 <= c <= 3):
        return False
    # LEFT side check
    if m > 0 and m < c:
        return False
    # RIGHT side check
    m_r = 3 - m
    c_r = 3 - c
    # same safety condition for right side
    if m_r > 0 and m_r < c_r:
        return False
    return True
# function to generate next possible states
def next_state(state):
    check=[] # list to store valid next states
    m,c,boat=state
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]  # possible moves of boat
(missionaries, cannibals)
    for i in moves:
        x, y = i
```

```
        # if boat is on left side, move people to right
        if boat == 'L':
            new = (m-x, c-y, 'R')
        # if boat is on right side, move people back to left
        else:
            new = (m+x, c+y, 'L')
        # add state only if it is valid
        if validity(new):
            check.append(new)
    return check
# function to apply Breadth First Search
def bfs():
    start = (3,3,'L')
    goal  = (0,0,'R')
    q = deque()
    q.append(start)
    visited = [] # list to store visited states
    visited.append(start)
    while q:
        current = q.popleft() # take first element from queue
        print(current)
```

```
        # check if goal state is reached
        if current == goal:
            print("Goal Reached")
            return

        # generate next valid states
        for n in next_state(current):
            if n not in visited:
                visited.append(n)
                q.append(n)

# calling BFS function
bfs()
```

Output

2-Water Jug Problem

```
PS C:\Users\gajay> python -u "c:\Users\gajay\Desktop\tempCodeRunnerFile.py"
Target is to get 4 litres in any one jug

Current State:
Jug X: 0 litres
Jug Y: 0 litres

Select Operation:
1. Fill Jug X
2. Fill Jug Y
3. Empty Jug X
4. Empty Jug Y
5. Pour Jug X into Jug Y
6. Pour Jug Y into Jug X
7. Exit
Enter choice: 1

Current State:
Jug X: 5 litres
Jug Y: 0 litres
```

```
Enter choice: 5

Current State:
Jug X: 2 litres
Jug Y: 3 litres
```

```
Enter choice: 4

Current State:
Jug X: 2 litres
Jug Y: 0 litres
```

```
Enter choice: 5

Current State:
Jug X: 0 litres
Jug Y: 2 litres

Select Operation:
1. Fill Jug X
2. Fill Jug Y
3. Empty Jug X
4. Empty Jug Y
5. Pour Jug X into Jug Y
6. Pour Jug Y into Jug X
7. Exit
Enter choice: 1

Current State:
Jug X: 5 litres
Jug Y: 2 litres
```

```
Enter choice: 5

Current State:
Jug X: 4 litres
Jug Y: 3 litres
Goal achieved
```

Missionaries and Cannibals

```
PS C:\Users\gajay> python -u "c:\Users\gajay\Desktop\Cannibals_missinary.py"
(3, 3, 'L')
(3, 2, 'R')
(3, 1, 'R')
(2, 2, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 2, 'L')
(0, 0, 'R')
Goal Reached
PS C:\Users\gajay>
```

Result Analysis

The Two Water Jug problem successfully demonstrates how user-selected operations lead to different state transitions until the desired amount of water is obtained. The program correctly updates the water levels after each action and stops execution when the target condition is satisfied in any jug.

In the Missionaries and Cannibals problem, only safe and valid states are explored during execution. The applied constraints prevent missionaries from being outnumbered on either bank. The use of systematic state exploration ensures that the goal state is reached without violating any conditions.

Viva Questions

Q1. Why is BFS preferred over DFS here?

BFS is preferred because it explores all possible states level by level and guarantees that the solution found is the shortest and safest one. In the Missionaries and Cannibals problem, BFS avoids going deep into unsafe or repetitive states, whereas DFS may get stuck in loops or explore invalid paths first.

Q2. What happens if boat capacity is increased to 3?

If the boat capacity is increased to 3, more people can be transferred at once, which reduces the

number of steps required to reach the goal. The problem becomes simpler, and additional valid states become possible. However, safety constraints must still be checked to ensure missionaries are not outnumbered on either bank.

Q3. Can the water jug problem be solved using BFS instead of manual input? How?
Yes, the water jug problem can be solved using BFS by representing each state as the amount of water in both jugs. All possible operations such as filling, emptying, and pouring can be treated as transitions between states. BFS can then be used to explore states systematically until the target amount is reached.

Q4. How do constraints prune invalid states**?**
Constraints help eliminate unsafe or invalid states early during execution. In the Missionaries and Cannibals problem, states where missionaries are outnumbered by cannibals are discarded. This reduces unnecessary exploration and ensures that only safe states are considered during the search process.

Conclusion

This experiment helped in understanding classical Artificial Intelligence problems through practical implementation. The Water Jug problem clarified the concept of state representation and how valid operations affect the system state through user interaction. The Missionaries and Cannibals problem provided insight into constraint satisfaction and safe state checking using systematic search. Overall, the experiment improved logical thinking and strengthened the understanding of structured problem-solving approaches in Artificial Intelligence.